```
        await next();
    }
}
```

This example takes the `HttpContext` as a parameter, but it also takes a `Func` that returns a `Task`, representing the next middleware in the pipeline. If the request includes a query string parameter named `stop`, the middleware short-circuits the pipeline and writes `Stop the world!` to the response. No other middleware is invoked. Otherwise, it invokes the `Func<Task>` that was passed in, passing control on to the next middleware. Middleware that passes control to the next component in the pipeline is registered with the `Use` method:

```
app.Use(PassThroughMiddleware);
```

Once again, this middleware can be written as an inline lambda.

Listing 2.12 Registering middleware inline with the `Use` method

```
app.Use(async (context, next) =>
{
    if (context.Request.Query.ContainsKey("stop"))
    {
        await context.Response.WriteAsync("Stop the world");
    }
    await next();
});
```

You can add code to run after control has been passed to the next middleware by placing it after `await next()`. Assuming that no other middleware short-circuits the pipeline, any logic you place there will be executed as the pipeline reverses its direction back to the web server. You might want to do this to include logging, for example.

Listing 2.13 Performing a function after other middleware has been invoked